Inference Regarding β_1

Within the context of this simple model, the primary objective is to perform inference regarding β_1, since it captures information regarding the relationship (or lack thereof) between the response and the single predictor.

Here are some key results in this direction. Keep in mind that these are all true under the assumptions put forth for the simple linear regression model.

- $\widehat{\beta}_1$ is an unbiased estimator of β_1.
- The variance of $\widehat{\beta}_1$ is

$$V(\widehat{\beta}_1) = \frac{\sigma^2}{(n-1)s_x^2} \approx \frac{\widehat{\sigma}^2}{(n-1)s_x^2}.$$

  This means that the standard error is

$$\mathrm{SE}(\widehat{\beta}_1) \approx \frac{\widehat{\sigma}}{s_x\sqrt{(n-1)}}.$$

- Suppose that the true value of the slope is β_{10}. Then

$$T = \frac{\widehat{\beta}_1 - \beta_{10}}{\mathrm{SE}(\widehat{\beta}_1)}$$

assuming ϵ_i are iid Normal

  has the t-distribution with $(n-2)$ degrees of freedom. Of course, when n is sufficiently large, T will be approximately standard normal.

$t = \hat{\beta}_1 / \mathrm{SE}(\hat{\beta}_1)$

Exercise: Construct a $100(1-\alpha)\%$ confidence interval for β_1.

Assuming ϵ_i are iid Normal$(0, \sigma^2)$

$$P\left(\underline{-t_{\alpha/2, n-2}} \le T \le \underline{t_{\alpha/2, n-2}}\right) = 1 - \alpha$$

$$P\left(-t_{\alpha/2, n-2} \le \frac{\hat{\beta}_1 - \beta_{10}}{SE(\hat{\beta}_1)} \le t_{\alpha/2, n-2}\right) = 1 - \alpha$$

Find that $\hat{\beta}_1 \pm t_{\alpha/2, n-2}\, SE(\hat{\beta}_1)$

is a $100(1-\alpha)\%$ CI for β_1

when n is large, use $z_{\alpha/2}$ instead of $t_{\alpha/2, n-2}$

In fact, when n large, version of CLT gives us

$\hat{\beta}_1 \approx$ Normal assuming that ϵ_i are independent

$$\hat{\beta}_1 = \sum_{i=1}^{n} w_i Y_i$$

A common (overused) objective is to test for evidence if one can claim that $\beta_1 \neq 0$. It is important to be mindful of the following fact: If the confidence interval for β_1 includes zero, this is often not because $\beta_1 = 0$, but instead because the standard error for $\widehat{\beta}_1$ is too large to detect the deviation from zero.

⋆ **Exercise:** How can the standard error of $\widehat{\beta}_1$ be decreased?

$$SE(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{(n-1)s_x^2}}$$

larger sample (larger n) $\Rightarrow$ more "power" to detect deviations from 0

One of the most common mistakes made when constructing models is to blindly remove a predictor from the model solely because there is not strong evidence that its cofficient does not equal zero. Model building (the process of deciding which covariates to include) should comprise many factors, including our knowledge of the situation, the importance that we are placing on constructing a **parsimonious** model, **and** the results of formal statistical inference procedures.

The Coefficient of Determination, R^2

When working with regression models it is very common to hear references to "the R^2 for the model."

This is also called the **coefficient of determination**.

The idea is simple: R^2 equals the proportion of the sum of squared response which is accounted for by the model that has been fit, relative to the model with no covariates.

In particular:

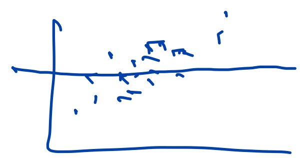
fitted values

$$R^2 = 1 - \left[\underline{\sum_{i=1}^{n} (y_i - \widehat{y}_i)^2} \Big/ \underline{\sum_{i=1}^{n} (y_i - \overline{y})^2} \right]$$

$\hat{\beta}_0 = \bar{y}$

Note that $0 \leq R^2 \leq 1$.

$\hat{\beta}_1 = 0$ $\quad$ $\hat{\beta}_0 = \bar{y}$



In the case of simple linear regression (a single predictor), R^2 does indeed equal the sample correlation (r) squared.

$$\sum_{i=1}^{n} \left(y_i - (\beta_0 + \underbrace{\beta_1 x_i}_{})\right)^2$$

R^2 is overused. In particular, note that it is possible for R^2 to be close to one, but the model be a terrible fit. And it is possible for a great-fitting model to have R^2 close to zero. See Figure 9.

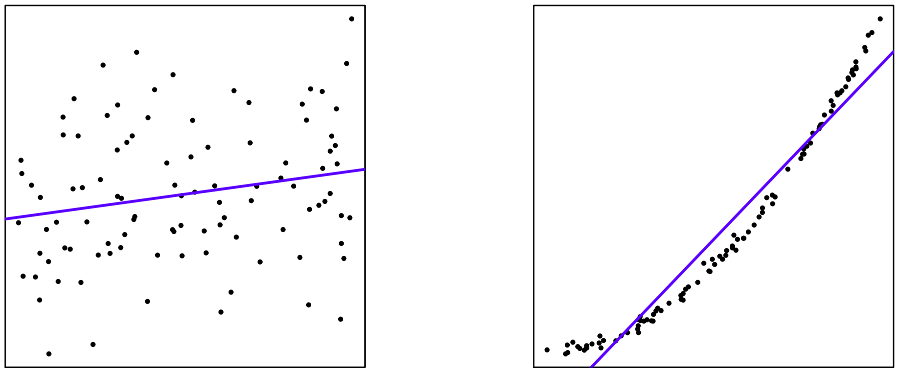


Figure 9: The scatter plot on the left has $R^2 \approx 0.05$, while that on the right has $R^2 \approx 0.95$. The data in the left plot were simulated from the normal linear model.

In short, R^2 should not be reiled upon as a diagnostic to judge the quality of the model fit. **If** the model is a good fit, **then** R^2 says something about the **predictive power** of the model.

⋆ **Exercise:** Suppose that, by mistake, each observed pair (response and predictor) is included twice in the analysis. What is the effect on $\widehat{\beta}_i$, the t-statistics, and R^2?

$t = \hat{\beta}_i / SE(\hat{\beta}_i)$

$\hat{\beta}_0$ and $\hat{\beta}_1$ remain the same

$$SE(\hat{\beta}_1) = \hat{\sigma} / s_x\sqrt{n-1} \qquad RSS = \sum (y_i - \hat{y}_i)^2$$

$$\hat{\sigma}^2 = \frac{RSS}{n-2} = \frac{2\,RSS_{true}}{2n_{true}-2} \approx \frac{RSS_{true}}{n_{true}-2} = \hat{\sigma}^2_{true}$$

s_x^2 stays (almost) the same

But, $\sqrt{n-1}$ becomes $\sqrt{2n_{true}-1}$, i.e. about $\sqrt{2}$ times larger

So, the SE of $\hat{\beta}_1$ is incorrectly divided by $\sqrt{2}$

The t-stat is incorrectly multiplied by $\sqrt{2}$

R^2 remains the same, both numerator and denominator of its fraction are multiplied by 2

$$s_x^2 = \frac{1}{n-1}\underbrace{\sum (x_i - \bar{x})^2}$$

Part 7: Multiple Regression

In this part, we will extend from the simple linear regression model to **multiple regression**.

These models comprise the familiar linear models that take the form

$$Y = \beta_0 + \sum_{j=1}^{p} \beta_j x_j + \epsilon$$

where ϵ are assumed to be uncorrelated with mean zero and variance σ^2.

Assuming that the ϵ are normally distributed brings additional nice properties, but is generally not essential.

We will go through the fundamental steps of fitting such a model via an example.

Example: Factor Models

References to the "beta" for a stock derive from the slope of the regression line fit to a scatter plot where excess return for that equity is the response, and excess market return is the predictor. Models of this form are an important component of **Capital Asset Pricing Model (CAPM)**.

The term **Factor Model** is used to indicate any linear model for excess return on an equity. The predictors in the model are the **factors**.

Quoting Ruppert, examples of factors include

1. excess returns on the market portfolio beyond the risk-free return
2. growth rate of the GDP;
3. interest rate on short term Treasury bills or changes in this rate;
4. inflation rate or changes in this rate;
5. interest rate spreads;
6. return on some portfolio of stocks;
7. the difference between the returns on two portfolios.

The CAPM only includes the factor for excess market return.

The **Fama-French Three Factor Model** adds two factors.

The first is **small minus big (SMB)**, the difference in returns between portfolios of small and large stocks.

The second is **high minus low (HML)**, the difference in returns between portfolios of high and low book-to-market stocks.

Kenneth French maintains a web site with data on the three factors

http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

This Python function allows for easy access to the daily factor data.

```
In [15]: import pandas_datareader.data as web
         import datetime, re, copy

         def getFamaFrench(from_date, to_date):

             three_fac = web.DataReader(\
                 "F-F_Research_Data_Factors_daily",
                 "famafrench")
             f = copy.copy(three_fac[0])
             f.rename( columns={c:re.sub(r'[0-9\-\s]','',c)\
                 for c in f.columns}, inplace=True)

             datetime.datetime.strptime(from_date, "%Y-%m-%d")
             datetime.datetime.strptime(to_date, "%Y-%m-%d")

             out = f[(f.index > datetime.datetime.strptime\
                 (from_date, "%Y-%m-%d")) &
                     (f.index < datetime.datetime.strptime\
                 (to_date, "%Y-%m-%d"))]

             return out
```

An example, and the first few rows of the data returned are as follows. (These factors are periodically revised, and hence the exact numbers may be different at the present time. This will affect the subsequent analysis, as well.)

```
In [16]: ffhold = getFamaFrench(from_date="2016-1-1",
                                to_date="2016-6-30")

         ffhold.iloc[0:10,]
```

```
Out[16]:              MktRF   SMB   HML   RF
         Date
         2016-01-04   -1.59 -0.84  0.53  0.0
         2016-01-05    0.12 -0.22  0.02  0.0
         2016-01-06   -1.35 -0.12 -0.01  0.0
         2016-01-07   -2.44 -0.28  0.08  0.0
         2016-01-08   -1.11 -0.48 -0.03  0.0
         2016-01-11   -0.06 -0.64  0.34  0.0
         2016-01-12    0.72 -0.40 -0.80  0.0
         2016-01-13   -2.67 -0.70  0.79  0.0
         2016-01-14    1.65 -0.01 -0.39  0.0
         2016-01-15   -2.14  0.42 -0.21  0.0
```

Let's get the data for PNC for the same time period. The Python package `yfinance` makes this simple.

```
In [17]: import yfinance as yf

         PNC = yf.download("PNC", "2016-01-01",
                 "2016-06-30")
```

```
[*********************100%***********************]  1 of 1 down
```

Now find the excess returns for PNC. Note that the Fama-French data file includes the risk free rate, with name `RF`:

```
In [18]: ffhold['PNCexret'] = \
          100*PNC['Adj Close'].pct_change() - ffhold['RF']
```

The first row does not have a return, so remove it.

```
In [19]: ffhold = ffhold[1:]
```

The three factor model we fit is

$$\text{PNC.Ex.Ret.} = \beta_0 + \beta_1 \,\text{Mkt.Ex.Ret.} + \beta_2 \,\text{SMB} + \beta_3 \,\text{HML} + \epsilon$$

A Python command for fitting a linear regression is `ols` in package `statsmodels`.

```
In [20]: import statsmodels.api as sm

         ff3modPNC = sm.OLS.from_formula\
           (formula="PNCexret ~ MktRF + SMB + HML",
              data=ffhold).fit()
```

The key information from the fit of the model is stored in the object `ff3modPNC`. For example, to see the parameter estimates and their standard errors for the estimators:

```
In [21]: ff3modPNC.params
```

Out[21]:

`Intercept    -0.151925` $\hat{\beta}_0$
`MktRF         1.215999` $\hat{\beta}_1$
`SMB          -0.011464` $\vdots$
`HML           0.798166`
`dtype: float64`

```
In [22]: ff3modPNC.bse

Out[22]: Intercept    0.070081
         MktRF        0.070531
         SMB          0.139419
         HML          0.120677
         dtype: float64
```

You can obtain the residuals from `ff3modPNC.resid` and the fitted values from `ff3modPNC.fittedvalues`.

Look at `ff3modPNC.summary()` for a more complete summary.

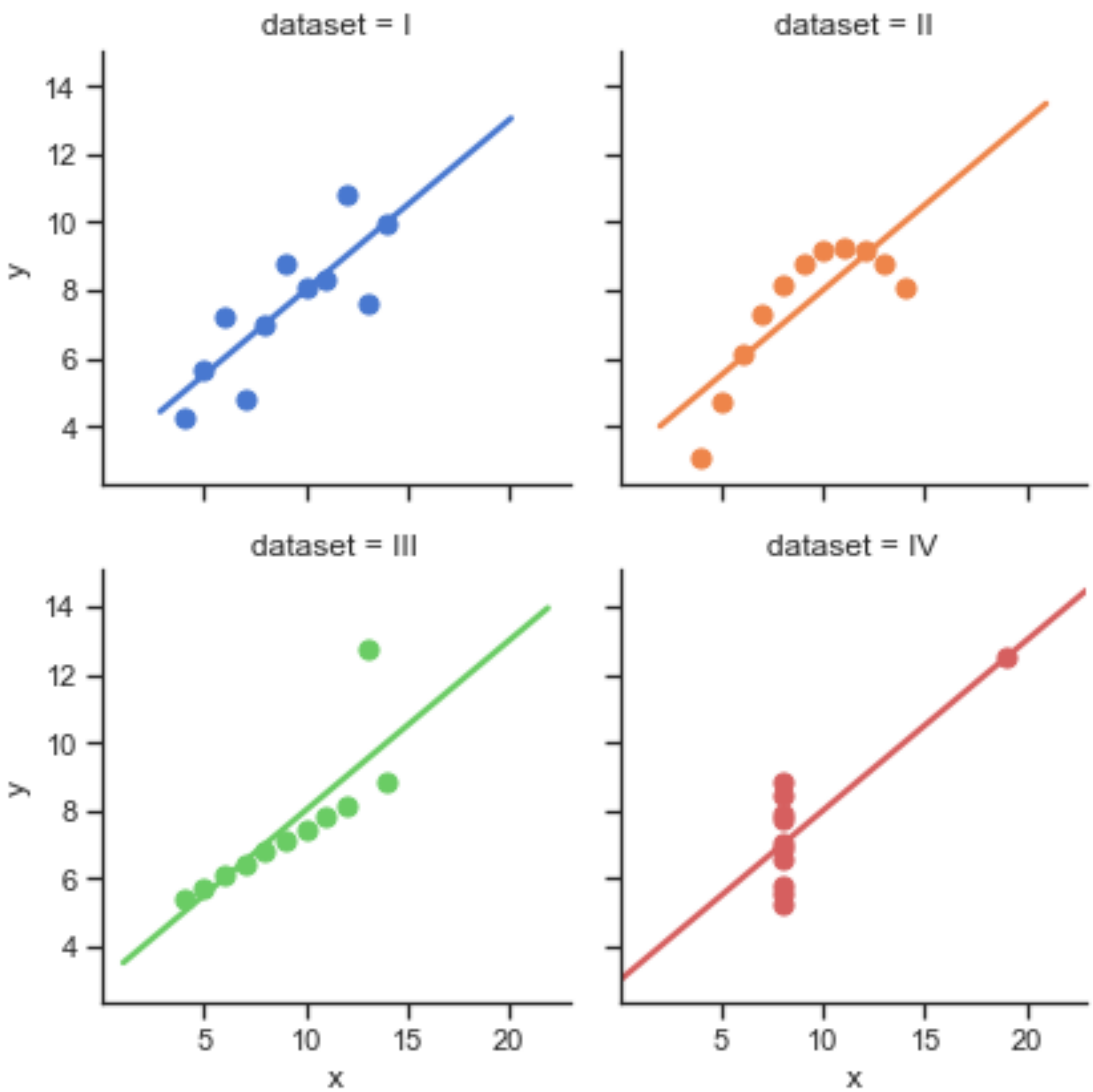
Diagnostic Plots

The figure above shows four scatter plots constructed by Francis Anscombe for a 1973 research article. These are called **Anscombe's Quartet**. This is, in part, based on https://seaborn.pydata.org/examples/anscombes_quartet.html.

```
In [23]: import seaborn as sns
         sns.set(style="ticks")

         ansdf = sns.load_dataset("anscombe")

         hold = sns.lmplot(x="x", y="y", col="dataset",
                    hue="dataset", data=ansdf, col_wrap=2,
                    ci=None, palette="muted", height=3,
                    scatter_kws={"s": 50, "alpha": 1})
```


dataset = I
dataset = II
dataset = III
dataset = IV
y
x
4
6
8
10
12
14
5
10
15
20

Exercise: Find the sample mean and standard deviation for both x and y in all four cases, to find the correlation between x and y in each case, and then use that to find the slope of the regression line for each.

For all 4 data sets, these summary measures are the same

Exercise: This can be taken a step further: We could find that the residual sum of squares in all four cases is exactly the same. What does this imply about the inference for β_1 in all four cases?

$SE(\hat{\beta}_1)$ is same, so t-stat is same, so judgements of "significance" are same

Exercise: What is the message we take away from this example?

Cannot use these summaries to assess model fit.

Only in case I is the model a good fit.

A very useful diagnostic plot is that of the **residuals versus the fitted values**. There are a wide range of models for which this is an invaluable tool.

Exercise: If the assumptions of the model are correct, what should we see in the plot of residuals versus fitted values?

There should be no remaining pattern. The residuals should show "random" scatter around the regression line/plane.

Exercise: Use the code below to explore the plot of residuals versus fitted values in the case of Anscombe's Quartet. Comment on the results.

In Case I, see no relationship in plot, indicating good fit

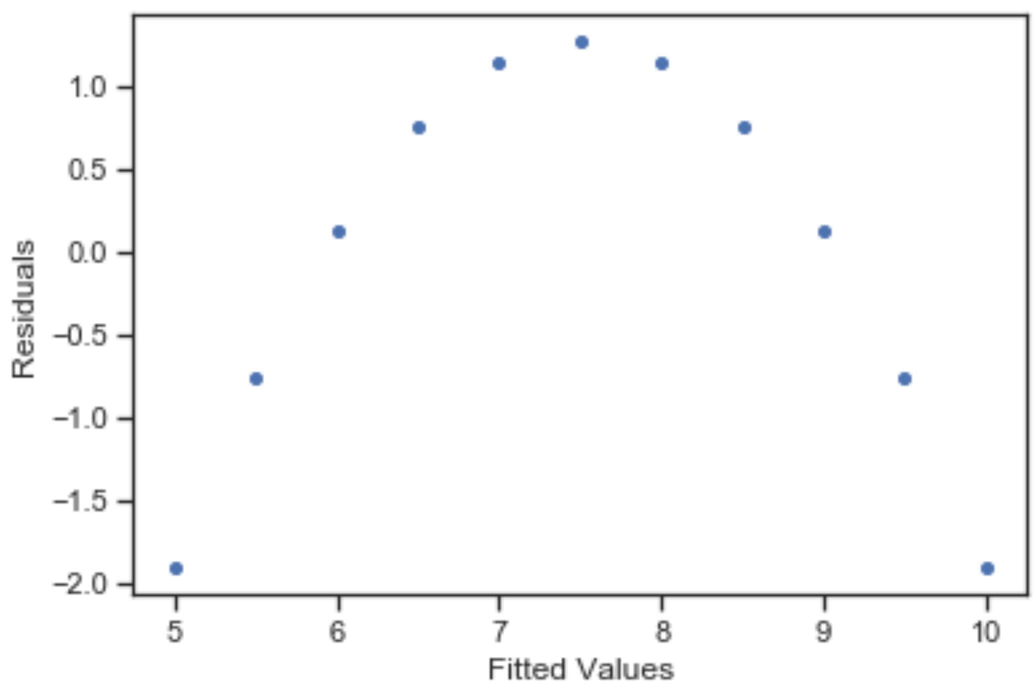
In Case II, pick up clear nonlinearity.

```
In [24]: ansgrp = ansdf.groupby(ansdf.dataset)
         onegrp = ansgrp.get_group("II")

         ansfit = sm.OLS.from_formula\
           (formula="y~x", data=onegrp).fit()

         import matplotlib.pyplot as plt

         plt.scatter(ansfit.fittedvalues,
             ansfit.resid, s=16)
         plt.xlabel("Fitted Values")
         plt.ylabel("Residuals")
         plt.show()
```



Exercise: Your colleague says, "I found the correlation between the residuals and fitted values to be equal to zero, so there is clearly no relationship between the residuals and fitted values. So, the model must be fine." Do you agree with this logic? No.

$Corr(\underset{\sim}{\hat{y}}, \underset{\sim}{\hat{\epsilon}}) = 0$ when β_0 included in model

$$Cov(\underset{\sim}{\hat{y}}, \underset{\sim}{\hat{\epsilon}}) = \frac{1}{n-1} \sum (\hat{y}_i - \bar{\hat{y}})(\hat{\epsilon}_i - \underbrace{\bar{\hat{\epsilon}}}_{0})$$

$$= \frac{1}{n-1} \left[\sum \hat{\epsilon}_i \hat{y}_i - \sum \hat{\epsilon}_i \bar{\hat{y}} \right]$$

$$= \frac{1}{n-1} \left[0 - 0 \right]$$

$$= 0$$

The residuals are also often plotted against each of the predictors. Our assumption is that the irreducible error is "random scatter" around the regression line/plane. If this assumption is true, there should be no remaining pattern evident in these plots.